

MACHINE LEARNING COURSE PROJECT • 2026

MEMRES $\rightarrow$ CGAR

Script thuyết trình chi tiết

Agentic Python Dependency Resolution

Nhóm **The Fangs**

Huỳnh Trung Kiệt · Đào Sỹ Duy Minh · Trần Chí Nguyên

University of Science, VNU-HCM

Tổng thời lượng phần chính (slide 1–31): $\approx$ 30 phút 20 giây. Backup (B1–B8) chỉ dùng khi Q&A.

Phân bổ: Mở đầu 1'00" · P1 Bài toán 7'20" · P2 MEMRES 3'40" · P3 CGAR 9'00" · P4 Kết quả 6'10" · P5 Hạn chế 3'10".

Con số ($\approx$ X giây) ở mỗi slide là thời lượng nói mục tiêu. Ghi chú *in xám nghiêng* là tùy chọn, nói hay không đều được.

Mở đầu

≈ 1'00"

Slide 1 · Giới thiệu đề tài & nhóm

(≈35s)

Xin chào thầy và các bạn. Nhóm em là The Fangs, gồm Huỳnh Trung Kiệt, Đào Sỹ Duy Minh và Trần Chí Nguyên. Nhóm em sẽ trình bày phương pháp CGAR được cải tiến lên từ MEMRES để phù hợp hơn với môn học. Overview thì đây là một hệ thống tự động dựng lại môi trường Python cho những đoạn code cũ, bằng Multi-Agent LLM và Constraint Acquisition. Vấn đề hiện nay là: code năm ngoái chạy được, nhưng hôm nay cài lại thì break vì thư viện đã đổi phiên bản. Mục tiêu của nhóm em là muốn tìm lại đúng bộ dependency, thay vì bắt người dùng dò tay từng phiên bản.

Slide 2 · Mục lục bài trình bày

(≈25s)

Bài trình bày sẽ gồm bối cảnh, bài toán và các công trình liên quan cũng như MEMRES là phương pháp nền tảng. Và CGAR, đề xuất chính của nhóm. Cuối cùng là đánh giá thực nghiệm và những hạn chế cùng với hướng phát triển.

Phần 01 · Bài toán

≈ 7'20"

Slide 3 · Phần 01: Bài toán

(≈10s)

Đầu tiên là phần bài toán: bối cảnh, đề bài, và câu hỏi vì sao nó lại khó.

Slide 4 · Bối cảnh

(≈75s)

Trên GitHub có rất nhiều đoạn code ngắn được chia sẻ công khai gọi là gist Python. Vấn đề là trong số các gist đó thì có một số cái khi tải về để cài đặt thì nó hỏng. Wheel thì build fail giữa chừng.

Nguyên nhân điển hình là do:

- Code lẫn cả Python 2 và Python 3.
- Wheel không build được trên máy hiện tại.
- Hay là file requirements mất nên không biết để cài.
- Hoặc là API mà code gọi đã bị xóa ở version mới.

Mục tiêu của nhóm là tự động dựng lại một môi trường chạy được cho những đoạn mã Python đã chết này để mọi import đều OK, cũng như môi trường được restore đúng như lúc code còn active.

Slide 5 · Bài toán & yêu cầu đánh giá

(≈85s)

Đầu vào (input) của bài toán sẽ là một Python snippet, một gist cũ. Không có metadata, không có requirements.txt, và cũng chưa biết nó chạy Python version nào.

Output cần trả về ba thứ. Python version phù hợp. Danh sách pip packages cùng với version của

nó. Và tùy chọn thêm các apt-get libs nếu cần.

Mọi import phải chạy được trong Docker, và phải không có ImportError, không ModuleNotFoundError, không SyntaxError xảy ra. Có mấy hard requirements bắt buộc (*optional: nhấn màu đỏ nhé, kiểu nói cũng được không nói cũng được*)

Mọi thao tác đều phải chạy trong Docker. LLM phải là open-source, và dưới 10GB VRAM. Và nhóm em sử dụng model Gemma-2 9B qua Ollama giống paper của ban tổ chức để dễ so sánh. Về chi phí đánh giá, các tool đều dùng chung một ngân sách để công bằng: Docker build tối đa 180 giây, nhiều nhất mười lần build mỗi snippet, số bước suy luận logic ký hiệu là K solve tối đa 50 và mấy bước này không đụng tới Docker, chạy một worker, và tổng khoảng 1800 giây mỗi snippet. Metric chính là pass rate, phụ là thời gian trung bình mỗi snippet.

Những cái yêu cầu trên đều được ban tổ chức cuộc thi yêu cầu.

Slide 6 · The Dependency Gap

(≈75s)

Có ba loại khó chính.

- Thứ nhất là tên import gây hiểu lầm. Ví dụ trong code viết import cv2, nhưng tên gói trên PyPI lại là opencv-python. Hai cái tên khác hẳn nhau.
- Loại thứ hai là API biến mất. Ví dụ from scipy dot misc import imread. Cái imread này chỉ tồn tại tới scipy 1.2, đến scipy 1.3 là nó bị bỏ hẳn.
- Loại thứ ba là hiệu ứng domino (dependency hell), tức là khi buộc phải chọn scipy từ 1.2 trở xuống, thì kéo theo Python phải từ 3.7 trở xuống, rồi lại kéo theo numpy phải là bản cũ. Cả một chuỗi ràng buộc nối đuôi nhau.

Vì vậy dẫn đến phải tìm tổ hợp trong không gian siêu bự.

Slide 7 · Bùng nổ tổ hợp

(≈65s)

Trong không gian tìm kiếm thì chỉ có đúng duy nhất một tổ hợp làm cho mọi import chạy được. Bên cạnh đó mỗi lần thử một tổ hợp không hề nhẹ/nhanh, mà phải dựng nguyên một Docker image rồi build, tốn thời gian. Cho nên là việc brute force bằng Docker là điều không khả thi. Bắt buộc phải có một solver hoặc một agent tốt để tĩa bớt giữ lại một nhóm candidate rồi mới đem đi build. Thì đó chính là hướng đi của bài toán.

Slide 8 · Datasets

(≈75s)

Ở đây bọn em sử dụng 2 bộ dữ liệu là:

- Bộ thứ nhất là HG2.9K, đây là bộ in-distribution. Gồm 2891 gist Python thuộc loại khó, là hard subset của Gistable, được công bố ở ICSME 2018 và cũng là dataset mà ban tổ chức cung cấp. Đặc điểm của nó là file đơn lẻ, không có requirements, hay lẫn Python 2 với Python 3, dùng nhiều thư viện cũ, và phụ thuộc vào C/C++.
- Bộ thứ hai là GitChameleon, đây là bộ out-of-distribution. Gồm 328 snippet Python, mỗi cái kèm unit test thực thi; còn version ground-truth thì được giấu đi, resolver không được thấy. Nó xoay vào breaking changes giữa các version, tức là bạn phải dùng đúng API của đúng bản thư viện. Tụi em đã re-frame nó từ GitChameleon 2.0 để đo tính generalization của phương pháp

cũng như là

(note, không đọc: nguồn số closed-model ở slide 24 = GitChameleon 2.0, arXiv 2507.12367, KHÔNG phải bản v1 116 bài)

Slide 9 · Related Work: 3 hướng

(≈75s)

Thì trước đây có ba hướng chính đã được triển khai là:

- Hướng thứ nhất là Knowledge Graph. Tức là encode mối quan hệ giữa module và version thành một đồ thị, lấy dữ liệu từ Libraries.io hay PyPI, rồi truy vấn để suy ra dependency. Ưu điểm là cấu trúc rõ ràng và deterministic. Nhưng hạn chế lớn nhất là database lỗi thời rất nhanh.
- Hướng thứ hai là regex và log-parsing. Người ta Parse error log bằng regex để đoán ra dependency đang thiếu, rồi patch vào và build lại. Ưu điểm là nhẹ và bám theo runtime. Nhưng nó brittle (dễ nhạy cảm), chỉ cần log đổi định dạng là parser vỡ.
- Hướng thứ ba là LLM kết hợp với RAG. Dùng LLM để sinh ra giả thuyết, RAG để cập thêm PyPI metadata, Docker build phản hồi lại, rồi lặp đi lặp lại. Ưu điểm là adaptive và tận dụng được kiến thức pretrain. Nhưng nó thử sai 1 cách mù quáng, những cái thất bại không được chuyển thành tri thức để prune không gian. Và đó chính là khoảng trống mà bọn em đề xuất method này. Chính là học từ các lần build thất bại để thu hẹp không gian tìm kiếm.

Slide 10 · Related Work: bảng tổng hợp

(≈55s)

- Nhóm Knowledge Graph có PyEGo ở ICSE 2022 và ReadPyE ở TSE 2024.
- Nhóm log-parsing có PyDFix ở ISSTA 2021. Nhóm LLM đóng có GPT-4o và o1 năm 2024, rồi GPT-4.1 cộng RAG năm 2025.
- Nhóm LLM có PLLM (của ban tổ chức). Và gần nhất là SMT-LLM (co-competitor) ở FSE 2026, kết hợp Z3 với LLM. Nhóm Knowledge Graph chạm ngưỡng trên ở khoảng 47% trên HG2.9K. Rồi PLLM mở ra cái hướng mới, vượt lên khoảng 55%.

Phần 02 · MEMRES

≈ 3'40"

Slide 11 · Phần 02: MEMRES

(≈10s)

Phần hai: MEMRES, phương pháp nền tảng của nhóm, dựa trên ý tưởng memory-augmented retry, tức là retry có bộ nhớ.

Slide 12 · FSE AIWARE Winner

(≈10s)

Trước khi chuyển từ MEMRES sang CGAR, nhóm em muốn đặt một mốc nhỏ: đây không chỉ là project lớp, mà còn gắn với FSE AIWARE Challenge. Slide này là chứng nhận winner, để cho thấy bài toán nhóm đang làm có bối cảnh nghiên cứu và benchmark thật.

Slide 13 · MEMRES: Lookup-First, LLM-Last

(≈95s)

Ý tưởng cốt lõi của memres thì đơn giản, đó là cascade rẻ trước, đắt sau. Tức là ưu tiên rẻ và chắc thì sẽ thử trước, chỉ khi khó quá mới gọi tới LLM. Cụ thể pipeline gồm bốn stage chính, cộng thêm hai fast path để bắt nhanh các ca dễ. Điểm hay nằm ở memory store nhận phản hồi từ mỗi lần build, rồi push ngược vào pipeline. Nói cách khác, MEMRES nhớ được giữa các lần retry, lần sau không lặp lại sai lầm của lần trước. Kết quả trên HG2.9K, đạt 86.3%. Trên GitChameleon đạt 81.7%. So với baseline PLLM chỉ 44.8 và 65.5%. Nhưng về bản chất thì MEMRES chỉ là retry được ghi nhớ. Tuy nó cứu được nhiều lỗi, nhưng mới chỉ nhớ được là mình đã sai, chứ chưa thật sự suy luận được vì sao sai. Cho nên mỗi lần retry, nó vẫn khá là mù.

Slide 14 · Từ MEMRES đến CGAR: vá 3 lỗ hổng

(≈115s)

Từ đó đã vạch ra ba lỗ hổng. Và cách làm của nhóm là với mỗi lỗ hổng thì sẽ biến thành đúng một thành phần trong phương pháp cải tiến.

- Thứ nhất là có những snippet mà thật sự khó. 12.8% là các ca rơi vào tình huống API đã bị gỡ bỏ, hoặc package chỉ có bản source chứ không có wheel. Với các case này, Docker cứ ngồi build kệt hết thời gian thì mới xác nhận là thất bại dẫn đến tốn thời gian. Để vá, nhóm thêm Analyzer Agent, đọc log lỗi và parse nó thành constraint.
- Thứ hai là chi phí cho một lần lỗi quá cao. Trung bình MEMRES tốn tới 335 giây mỗi snippet, và các ca khó thì vẫn rất chậm. Để solve thì nhóm thêm hàm wheel_filter, chỉ thử những bản có sẵn wheel, bỏ qua các bản phải build từ source.
- Cuối cùng, cũng là quan trọng nhất, là MEMRES không tĩa không gian. Mỗi lỗi Docker bị coi như một blackbox, thất bại rồi thì thôi, chứ nó không được chuyển thành ràng buộc để loại hẳn cả một vùng lời giải. Để vá, nhóm đã thiết kế hệ ba tầng constraint, gồm HARD, SOFT, và UPPER bound. Gộp cả ba thành phần này lại, chuyển từ LLM đoán mò sang Multi-Agent kết hợp Constraint Acquisition, agent tự học ràng buộc từ chính phản hồi của verifier. Đó chính là CGAR.

Phần 03 · CGAR (trọng tâm)

≈ 9'00"

Slide 15 · Phần 03: CGAR

(≈10s)

Đây là phần CGAR mà nhóm đã trình bày ở trên.

Slide 16 · CGAR: ba agent

(≈75s)

CGAR gồm ba LLM agent, mỗi agent có một vai trò rõ ràng.

Agent thứ nhất là Planner. Nhiệm vụ của nó là chọn version cho từng package, ra một phương án lắp ghép cụ thể.

Agent thứ hai là Analyzer. Nó đọc lỗi Docker trả về, hiểu lỗi đó nói gì, rồi sinh ra một ràng buộc.

Agent thứ ba là Critic. Khi cả hệ không solve được, tức là thử mãi không được, thì Critic mới vào

cuộc, nó đề xuất đổi chiến lược, ví dụ đổi hẳn phiên bản Python.

Cả ba agent này xoay quanh một Docker build loop, tức là vòng lặp build và kiểm thử, và cùng dùng chung một Session Store, một bộ nhớ ràng buộc chia sẻ. Chi tiết prompt của từng agent em để ở phần backup ở cuối slide. Và tiếp theo em sẽ trình bày phần cơ chế.

Slide 17 · Multi-Agent Loop

(≈110s)

Với một vòng lặp chính thì bước một là Planner. Planner sẽ gọi `query_pypi` để lấy dữ liệu package, gọi `wheel_filter` để lọc bản có wheel, gọi `consult_llm` khi cần hỏi model, rồi gọi `solve` để ra một assignment version, tức là gán mỗi package một phiên bản cụ thể. Bước hai là Executor. (Lưu ý Executor là tool). Nó `build_docker` để dựng môi trường, rồi `run_import` để chạy thử import. Xong nó chờ phản hồi là build có OK không. Nếu OK thì Done, thoát vòng lặp. Nếu không OK thì sang bước ba, Analyzer. Analyzer gọi `parse_error` để bóc tách lỗi, `consult_llm` để hiểu ngữ cảnh, rồi `gen_constraint` để sinh ra một ràng buộc. Ràng buộc đó được thêm vào Session Store, và phân loại thành một trong ba nhóm: HARD, SOFT, hoặc UPPER. Nếu vẫn còn kẹt do bí thì mới tới bước bốn, Critic. Critic gọi `analyze_failures` để nhìn lại cái chuỗi thất bại, `suggest_strategy` để gợi ý hướng khác, ví dụ đổi Python. Điểm mấu chốt là mỗi lần build thất bại không hề phí. Vì nó sinh ra thêm một ràng buộc. Ràng buộc đó khiến Planner ở vòng sau né được cả một vùng version, chứ không chỉ né một điểm. Đây chính là thứ MEMRES cũng như các method trước không có.

Slide 18 · Miền giá trị D đến từ đâu

(≈90s)

Vậy Solver chọn version từ đâu ra? Thì câu trả lời là miền giá trị D, là tập ứng viên mà solver được phép chọn. Miền này được lọc trực tuyến từ PyPI qua một pipeline năm bước, và điều quan trọng là không tốn quota LLM.

Bước một, đọc import trong code, ví dụ thấy `scipy` với `numpy`, suy ra target Python là 3.7.

Bước hai, map tên import sang tên pip bằng một bảng tĩnh, ví dụ `scipy.misc` thì gói pip là `scipy`.

Bước ba, gọi HTTP tới `pypi.org` lấy file json của gói, chỉ là một request thuần, không LLM.

Bước bốn, với mỗi version, giữ lại nếu trường `requires_python` thỏa target Python và gói đó có sẵn wheel.

Bước năm, sắp xếp, ưu tiên bản có wheel trước, rồi semver giảm dần, và chỉ lấy tám ứng viên đầu mỗi gói.

Slide 19 · Tích lũy ràng buộc: ví dụ

(≈80s)

Tiếp theo là sẽ là 1 ví dụ cho thấy cơ chế tích lũy ràng buộc thu hẹp miền đó ra sao. Dòng code là `from scipy dot misc import imread`. Lần thử thứ nhất, solver build `scipy 1.7.3`. Kết quả là lỗi `ImportError`, thông báo `cannot import name imread`. Analyzer đọc lỗi này và học được một điều: hàm `imread` đã bị bỏ ở bản mới, nên `scipy` phải nhỏ hơn 1.3. Nó ghi một ràng buộc loại UPPER, `scipy` nhỏ hơn 1.3, vào Session Store. Lần thử thứ hai, solver tự chọn `scipy 1.2.3`, đây là bản cao nhất mà vẫn nhỏ hơn 1.3 và vẫn có wheel. Lần này build OK. Điểm đáng chú ý là gì. Chỉ sau đúng một lần fail, solver nhảy thẳng từ 1.7.3 về đúng 1.2.3, không dò mò qua từng bản ở giữa. Một lỗi build cụ thể đã biến thành một cận trên, và cận trên đó cắt luôn cả một khoảng version hổng, chứ không chỉ một điểm.

Slide 20 · Feedback-Guided Constraint Discovery

(≈100s)

Ý tưởng cốt lõi là: ràng buộc thật bị ẩn, thì mình sẽ không biết trước. Nó chỉ lộ ra qua một verifier hợp đen, gọi là f của A . Đưa một phương án A vào, nó trả về hoặc pass, hoặc một typed-error, tức lỗi có phân loại. Mình bắt đầu với tập ràng buộc C_0 rỗng, chưa biết gì cả. Mỗi lỗi trả về là một counterexample, và agent học thêm một ràng buộc từ đó. Đây là Constraint Acquisition, theo kiểu CEGIS, chứ không phải giải một CSP đã cho sẵn. Trạng thái tìm kiếm ở bước t gồm ba phần: tập biến X , miền giá trị D , và tập ràng buộc C ở thời điểm t . Trong đó C lớn dần theo thời gian. Có ba loại ràng buộc học được. HARD cấm vĩnh viễn version lỗi rõ ràng. SOFT cấm tạm, chỉ khi lỗi lặp lại từ hai lần trở lên. UPPER dạng version nhỏ hơn một cận trên, cấm cả một khoảng. Vòng lặp gói gọn: chọn phương án A thỏa C hiện tại, build, nếu fail thì học C mới và giải lại, tối đa năm mươi lần suy luận logic.

Slide 21 · Session-scoped Learning

(≈75s)

Còn một điểm nữa làm phương pháp CGAR khác biệt, đó là học theo phạm vi session. Một session ở đây là một repository, gồm nhiều snippet. Điểm cốt lõi nằm ở Session Store: nó giữ toàn bộ ràng buộc học được xuyên suốt các snippet trong cùng một batch, và không bị reset giữa các snippet. Nhờ vậy, ràng buộc học được từ snippet này được chia sẻ thẳng cho snippet khác trong cùng session. Nói cách khác, snippet trước dạy snippet sau tránh những vùng version đã hỏng. MEMRES thì xử lý mỗi snippet một cách độc lập, nên nó không có cơ chế tích lũy này.

Phần 04 · Kết quả

≈ 6'10"

Slide 22 · Phần 04: Kết quả

(≈10s)

Tiếp theo là đến phần kết quả.

Slide 23 · So sánh tổng thể

(≈95s)

Bảng này sẽ so sánh CGAR với toàn bộ các tool khác trên hai bộ dữ liệu.

CGAR đạt 87.1% trên HG2.9K và 83.2% trên GitChameleon, trung bình 22.3 giây mỗi snippet, và dùng model mở.

Hai tool cũ, PyEGo và ReadPyE, chỉ quanh 45 tới 47%.

Nhóm closed model đều dưới 60%, kể cả o1 tốt nhất cũng chỉ 51.2% trên GitChameleon.

PLLM, cũng là open tool, chỉ 44.8% và mất tới gần 370 giây mỗi snippet. Còn hai tool mạnh nhất trước CGAR là SMT-LLM 83.6% và MEMRES 86.3%.

Trên HG2.9K, ngân sách cho mọi tool là như nhau, tối đa mười lần build, mỗi lần 180 giây. CGAR hội tụ trong dưới năm build thay vì dùng hết ngân sách. Cho thấy CGAR vừa cao nhất, vừa nhanh nhất trong nhóm open tool.

Slide 24 · Phân rã lỗi trên HG2.9K

(≈85s)

Nhìn vào từng loại lỗi. Biểu đồ bên trái đếm số build fail theo loại lỗi, PLLM so với CGAR.

Ba nhóm lỗi lớn: SyntaxError 494 ca, NoMatchingDistribution 282 ca, AttributeError 83 ca, CGAR đều triệt tiêu hoàn toàn. ImportError giảm từ 433 xuống 372. Biểu đồ bên phải là tỷ lệ rescue theo từng loại lỗi mà PLLM đã fail. Rescue dao động từ 66.7% với FailedToRun, ca khó nhất, thường là lỗi native hoặc nền tảng, tới 97.1% với NameError. Cộng lại: PLLM fail 1596 snippet, sau CGAR chỉ còn 373, giảm 76.6%.

Slide 25 · GitChameleon: Open vs Closed

(≈95s)

Trên GitChameleon. GPT-4o 49.1%, Gemini 2.5 Pro 50.0%, o1 51.2%, đây là model closed tốt nhất trong nhóm. GPT-4.1 cộng RAG 58.5%. Các số closed này lấy từ báo cáo GitChameleon 2.0, do trên task code-gen, nên chỉ mang tính tham chiếu độ khó của benchmark. Rồi tới nhóm open: PLLM 65.5%, MEMRES 81.7%, và CGAR dẫn đầu với 83.2%. Cột bên phải đo khả năng tổng quát hóa. Cho thấy CGAR không chỉ tốt trên specific trên một bộ dữ liệu, nó chạy trên metadata PyPI thật nên chuyển bộ dữ liệu vẫn ổn định. Cho thấy việc thiết kế hệ thống là rất quan trọng.

Slide 26 · Tốc độ và Ablation

(≈85s)

Thời gian chạy trên GitChameleon, cả ba tool cùng ngân sách năm vòng Docker build. CGAR có median thấp nhất; PLLM và MEMRES đều chậm hơn. Biểu đồ bên phải là ablation trên 396 ca rescue. Full CGAR cứu được 71 ca. Bỏ session store còn 56, giảm 21%. Bỏ wheel filter còn 40, giảm 44%. Bỏ upper bound còn 23, giảm 68%. Vậy cả ba thành phần đều cần, và có thể thấy upper bound đóng góp lớn nhất. Khi một ca là bất khả thi, CGAR biết sớm: lúc fail chỉ chậm hơn lúc pass 1.31 lần, còn PLLM tới 2.20 lần.

Phần 05 · Hạn chế & Hướng phát triển

≈ 3'10"

Slide 27 · Phần 05: Hạn chế & Kết

(≈8s)

Hạn chế hiện nay của CGAR.

Slide 28 · Hard Floor & Giới hạn kiến trúc

(≈80s)

Đầu tiên là hard floor. Có 310 snippet, tức 10,7% của bộ dữ liệu, mà cả PLLM lẫn CGAR đều fail. Nhóm em cũng đã xem qua và phần lớn là được viết bằng Python2 (41.6%) mà Python 2 thì không còn wheel trên môi trường hiện đại. Tiếp theo, 25,8% là ImportError trên các package system hoặc private, kiểu idaapi hay appscript, những thứ không có trên PyPI công khai. Khoảng 13% là NoMatchingDistribution, tức package đó không tồn tại trên PyPI. Khoảng 19% còn lại là native-build vướng glibc, API đã bị gỡ.

Thì đây là phần không thể giảm được nữa, dưới ràng buộc là chạy trong Docker, chỉ dùng PyPI public, không patch source code, và tối đa mười lần retry. Nói cách khác, thì đây không phải problem của tool. Ngoài hard floor, còn bốn hạn chế về kiến trúc.

- Một, CGAR phụ thuộc PyPI live, nếu PyPI down hoặc rate-limit thì phải fallback về cascade

của MEMRES.

- Hai, hiện tại mới chỉ chạy được cho Python, chưa generalize sang JS, Rust hay Go.
- Ba, LLM Analyzer khá brittle với log lạ nằm ngoài dữ liệu pretrain của Gemma-2 9B.
- Bốn, session store mới chỉ single-worker, chưa hỗ trợ học federated cho nhiều người dùng.

Slide 29 · Hướng phát triển

(≈70s)

Từ những hạn chế đó, nhóm có bốn hướng để phát triển.

- Hướng thứ nhất là mở rộng đa ngôn ngữ.
- Hướng thứ hai là chia sẻ constraint mà vẫn bảo toàn riêng tư. Ý tưởng là snippet của user A khi fix xong sẽ giúp user B tránh đúng lỗi tương tự, mà không cần lộ code của ai.
- Hướng thứ ba là tinh chỉnh khả năng đọc lỗi của LLM. Fine-tune Gemma-2 trên dữ liệu đọc log, để nó phân loại lỗi và sinh constraint chính xác hơn.
- Hướng thứ tư là mở rộng đa mô hình. Chạy CGAR trên nhiều backbone mới hiện nay để chứng minh nó tốt cho nhiều loại mô hình.

Slide 30 · Tài liệu tham khảo

(≈8s)

Phần tài liệu tham khảo chính của nhóm em nằm ở đây.

Slide 31 · Video trình bày

(≈10s)

Nhóm cũng có chuẩn bị một video trình bày cách phương pháp hoạt động. Chỉ cần bấm vào thumbnail là xem được.

Slide 32 · Cảm ơn & Hỏi đáp

(≈15s)

Phần trình bày của nhóm đến đây là hết. Cảm ơn thầy đã lắng nghe.

Backup · chỉ dùng khi Q&A

Slide B1 · Filter & Solver: đầu vào đầu ra

(Q&A)

Đây là hai bước lõi.

CandidateGraphBuilder làm filter: nhận packages và python_version, gọi endpoint json của PyPI, lọc theo requires_python và wheel Linux, sort wheel trước mới nhất trước, giữ top 8.

ConstraintSolver nhận graph và store, prune theo constraint store, greedy chọn bản mới nhất còn khả thi, nếu combo bị loại thì duyệt kiểu odometer, tối đa 50 attempts thuần logic.

Cần nhớ: Ksolve bằng 50 chạy logic tính bằng micro giây, còn Kbuild nhỏ hơn hoặc bằng 10 là Docker thật, 180 giây mỗi lần, nên rất đắt. (*optional*)

Slide B2 · Cách các agent giao tiếp

(Q&A)

Hệ có 4 agent: Planner, Executor, Analyzer, Critic, trao đổi qua 6 luồng message. Mấu chốt: agent không gọi nhau trực tiếp, CGARResolver làm coordinator điều phối, còn ConstraintStore là bộ nhớ chung mang tri thức chéo snippet. Ví dụ: Analyzer ghi `add_upper_bound` cho scipy bản 1.3, sau đó Planner hỏi `get_upper_bound` và nhận lại 1.3.

Slide B3 · Cấu trúc dữ liệu Session Store

(Q&A)

Bản ghi lỗi là `InfeasibleRecord`, gồm `package`, `version`, `python_version`, `error_type` HARD hoặc SOFT, `error_signature`, `confidence` 1.0 hoặc 0.8, và `count`. Có ba store: `records`, `combo_records` và `upper_bounds`. Cơ chế: nhận log lỗi, normalize signature để cắt bỏ số dòng, địa chỉ bộ nhớ dạng 0x và đường dẫn, rồi `classify_error`. Regex bắt chuỗi `cannot import name X from PKG` là tự động `add_upper_bound`.

Slide B4 · Cơ chế gọi LLM

(Q&A)

Agent gọi LLM qua HTTP tới Ollama chạy Gemma-2 9B ở localhost cổng 11434. Budget theo agent: Planner 128 token mỗi plan step, Analyzer 128 token mỗi lần Docker fail, Critic 96 token khi có từ 3 lỗi cùng loại trở lên. `temperature` 0.3, `format json`, HTTP timeout 60 giây, retry một lần. Parse thất bại thì quay về rule fallback.

Slide B5 · Build loop & chống ảo giác

(Q&A)

Mỗi snippet reset state rồi loop tối đa 10 Docker attempt, pass thì break. Mỗi attempt đi 4 bước: Planner, Executor, Analyzer, Critic. Phần chống ảo giác: mỗi agent bị ràng ra khỏi tập giá trị hợp lệ của nó. Version Planner chọn phải nằm trong candidate graph. Type Analyzer trả phải là loại hợp lệ đã định nghĩa. Action Critic phải là `continue`, `switch_python` hoặc `mark_unfixable`. JSON parse fail thì regex salvage.

Slide B6 · Prompt của Planner

(Q&A)

Planner prompt: chọn đúng 1 version mỗi package sao cho dễ install và import trên Python pi, ưu tiên bản có wheel, tránh các bản đã fail. Nếu bản rule pick đã an toàn thì trả lại luôn. Output JSON gồm `assignment` và `reason`.

Slide B7 · Prompt của Analyzer

(Q&A)

Analyzer prompt phân loại lỗi thành HARD, SOFT hoặc UPPER. Luật trọng tâm: gặp `cannot import name X from PKG` thì gán UPPER và điền `upper_bound` bằng bản tốt cuối cùng trước khi X bị gỡ bỏ. Output JSON gồm `type`, `package`, `upper_bound` và `reason`.

Slide B8 · Prompt của Critic

(Q&A)

Critic prompt đề xuất pivot: `continue`, `switch_python` hoặc `mark_unfixable`. Luật: `SyntaxError` lặp dưới Python 3 mà chưa thử 2.7 thì `switch_python` sang 2.7; `ImportError` lặp trên 4 lần thì `mark_unfixable`; quá 8 attempt lỗi lẫn lộn cũng `mark_unfixable`. Output JSON gồm `action`, `target` và `reason`.